第6章 第1节 为什么需要Function Calling

- 来源: https://class.imooc.com/lesson/5275#mid=104925
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第6周 Function Calling：工具调用机制
- 模块: Function Calling的本质
- 课节: 图文 1-1 为什么需要Function Calling

1-1 为什么需要Function Calling

第6章 第1节 为什么需要Function Calling

> [!TIP] 阅读指南 前面学习的大模型只能输出文本，无法完成实际操作。这一节引入Function Calling的概念——让AI不仅能"说"，还能"做"。从电影《Her》的畅想出发，分析传统LLM的局限，介绍Function Calling的工作模式、能力范围和限制，最后讨论它与RAG的关系。

1.1 从电影《Her》说起

2014年奥斯卡获奖影片《Her》，讲述了主人公西奥多与人工智能萨曼莎（Samantha）的故事。影片里有大量人与人工智能的对话。

当西奥多说："帮我整理一下今天的邮件。"萨曼莎立刻读取邮箱，分类整理，删除垃圾邮件。

当西奥多说："帮我查一下去海边的路线。"萨曼莎调用地图系统，规划最佳路线，还贴心地查询了天气。

当西奥多说："帮我把这封信发给出版社。"萨曼莎直接发送邮件，并设置了后续提醒。

萨曼莎不只会"陪西奥多聊天"，更会"帮他做事"。而前面章节学到的AI，更像是"只会对话的朋友"——它能理解用户的心情、回答问题，但无法真正完成任务。Function Calling 就是让AI从"聊天伙伴"变成"智能助手"的关键技术。

1.2 传统LLM的困境

目前学过的大模型有一个共同的局限：只能输出文本，无法执行操作。换句话说，它们只能"说"，不能"做"。下面两个场景可以直观感受到这种局限。

比如查询天气，当用户问"明天上海天气怎么样？"时，传统 LLM 会这样回答：

```
抱歉，我的知识截止到2023年10月，无法提供实时天气信息。
建议您查看天气预报网站或APP。

1
2
```

明明有天气 API 可用，AI 却帮不上忙。再比如设置提醒，当用户说"提醒我明天下午 3 点开会"时，它也只能这样回答：

```
好的，请您在日历应用中设置明天下午3点的会议提醒。

1
```

AI 只会"教用户怎么做"，而不是"帮用户做"。

为什么会这样？根源在于传统 LLM 的工作模式被困在"文本输入 → 文本输出"这样一条封闭的回路里。

它的知识停留在训练截止那一刻——查不到今天的天气，也看不到此刻的股价。

它没有执行器——没法真的把邮件发出去，也没法替你按下提醒的闹钟。

它更没有伸向外部世界的手——专业计算工具、数据库、文件系统，它都摸不到。

它就像一个只能说话、没有手脚的助手。

它就像一个只能说话、没有手脚的助手

1.3 Function Calling：赋予AI行动能力

Function Calling 让LLM不仅能"说"，还能"做"。通过调用外部函数、工具或API，大模型能够完成需要实际操作的任务——相当于给它装上了"手脚"。

它的工作模式是这样的：

```
用户输入（文本）
    ↓
LLM分析：需要什么工具？
    ↓
调用外部函数（获取数据/执行操作）
    ↓
LLM基于函数结果生成回答
    ↓
输出最终答案（文本）
```

下面用一段伪代码快速体验一下，场景是查询明天上海的天气：

```
# ═══════════════════════════════════
#  场景：查询明天上海的天气
# ═══════════════════════════════════

# 1. 用户提问
user_input = "明天上海天气怎么样？"

# 2. 定义可用的工具
tools = [
    {
        "name": "get_weather",
        "description": "获取指定城市的天气信息",
        "parameters": {
            "city": "城市名称",
            "date": "日期（如：明天、后天、2025-12-15）"
        }
    }
]

# 3. LLM分析并决定调用工具
response = llm.chat(
    message=user_input,
    tools=tools  # 告诉LLM有哪些工具可用
)

# 4. LLM返回：我需要调用get_weather
response = {
    "tool_call": {
        "name": "get_weather",
        "arguments": {
            "city": "上海",
            "date": "明天"
        }
    }
}

# 5. 执行实际的函数调用
def get_weather(city, date):
    # 这里调用真实的天气API
    api_result = call_weather_api(city, date)
    return api_result

result = get_weather("上海", "明天")
```

```
# result = {
#     "city": "上海",
#     "date": "2026-01-16",
#     "temperature": "15-22℃",
#     "weather": "晴",
#     "wind": "北风3级"
# }

# 6. 把函数结果返回给LLM
final_answer = llm.chat(
    message=user_input,
    tool_result=result
)

# 7. LLM生成最终回答
print(final_answer)
# "明天上海晴天，气温15-22℃，北风3级，
#  是个适合出行的好天气！建议穿薄外套。"
```

在这一流程中，LLM 的角色发生了本质变化。它先理解意图并识别出需要调用的工具（get_weather），随后精准地提取出查询所需的参数（上海、明天）。在实际调用天气 API 获取真实数据后，LLM 最终基于这些实时信息生成准确的自然语言回答。

完整流程可以用下图表示：

1.4 Function Calling能做什么

理论上，只要能写出函数，LLM 就能调用它。这意味着无论是天气、地图、数据库等第三方 API，还是自行实现的文件操作、数据处理等业务逻辑，甚至是对硬件设备的控制和系统命令的执行，都可以交给 LLM 来驱动。

但实践中有几个限制需要注意。首先是安全风险，不要把危险操作暴露给 LLM 调用——如果 LLM 误解了用户的意图（例如将"清理一下"理解为删除所有数据），或者遭遇 Prompt 注入攻击，执行了恶意代码或越权转账，后果将是灾难性的。因此需要遵循严格的安全原则：对于查询天气、搜索信息等只读操作，可以放心交给 AI；但对于删除数据、修改配置等破坏性操作，以及转账、支付等资金相关操作，必须引入人工确认机制；至于修改密码、授权等敏感权限操作，绝不应让 LLM 直接调用。

其次是成本与性能。LLM 调用本身和高昂的第三方 API 费用带来成本压力；而模型判断加上 API 响应所需的时间，使得它在毫秒级响应的实时性场景中并不适用；此外 LLM 仍存在准确性风险，可能会误判函数或提取错误参数，因此必须做好完善的错误处理。

说完限制，再来看实践中的典型应用。下图按风险级别由低到高，展示了 Function Calling 的适合与不适合场景：

```
Function Calling 应用场景
                    (按风险级别从低到高)

【安全区】只读操作 - 放心使用
  ◆ 查询天气          ◆ 搜索信息          ◆ 数学计算
  ◆ 股票行情          ◆ 新闻资讯          ◆ 数据库查询
  ◆ 地图导航          ◆ 汇率转换          ◆ 单位换算

          ↓ 风险逐渐增加

【谨慎区】写操作 - 需要确认机制
  ▲ 发送邮件          ▲ 设置提醒          ▲ 创建日程
  ▲ 文件读取          ▲ 保存笔记          ▲ 生成报表
  ▲ 数据分析          ▲ 智能推荐          ▲ 内容生成

          ↓ 风险持续上升

【危险区】破坏性操作 - 严格限制或禁止
  ✗ 删除数据          ✗ 修改配置          ✗ 执行代码
  ✗ 转账支付          ✗ 修改密码          ✗ 授权操作
  ✗ 系统命令          ✗ 数据库写入        ✗ 文件删除

【核心原则】
  可逆操作  ──→  可以自动执行
  不可逆操作 ──→  必须人工确认
  高风险操作 ──→  严格限制或禁止
```

1.5 与RAG的关系

RAG 和 Function Calling 都是让大模型突破自身局限的手段，但解决的问题不同。

RAG的核心是让AI拥有专属知识库，解决的是"知识"问题：

```
让AI拥有专属知识库
→ 回答专业问题
→ 解决"知识"问题
```

典型场景包括企业知识库问答、技术文档查询、法律条文检索，核心能力是检索 + 生成。

本章学的Function Calling，核心是让AI拥有行动能力，解决的是"能力"问题：

```
让AI拥有行动能力
→ 执行实际操作
→ 解决"能力"问题
```

典型场景包括调用API获取数据、操作系统和应用、控制外部设备，核心能力是判断 + 调用。

两者结合

把两者结合起来，就能组成真正的AI助手。以一个智能客服场景为例：

真正的AI助手

```
用户："我想退货"

流程：
1. RAG检索退货政策（知识）
2. Function Calling查询订单信息（能力）
3. Function Calling创建退货申请（能力）
4. 基于政策+订单信息生成回答（综合）

回答：
"您的订单#12345符合7天无理由退货政策。
已为您创建退货申请，退货单号：RT789。
请在3天内将商品寄回，预计5个工作日退款到账。
需要我帮您预约快递上门取件吗？"
```

1.6 下一节预告

既然 Function Calling 如此强大，大模型究竟是如何实现这一点的？它是如何做到在没有运行代码的情况下，精准判断该调用哪个函数并提取出正确参数的？下一节，我们将深入探索 Function Calling 的工作原理。

1.7 ■ 学点英语

中文	English	音标	说明
函数调用机制	Function Calling	/ˈfʌŋkʃn ˈkɔːlɪŋ/	让LLM不仅能输出文本，还能调用外部函数执行实际操作
安全分级	Security Classification	/sɪˈkjʊərəti ˌklæsɪfɪˈkeɪʃn/	按操作风险程度将FC能力分为安全区/谨慎区/危险区
只读操作	Read-only Operation	/riːd ˈəʊnli ˌɒpəˈreɪʃn/	不改变系统状态的数据查询类操作，可放心交给AI
工具调用	Tool Call	/tuːl kɔːl/	LLM以结构化JSON形式返回的、指示代码执行特定函数的指令

图片